

05：阶段 0——精确端点局部接入

1. 阶段 0 要解决什么

压缩图会删除区域内部节点。如果查询起点或终点是内部节点，旧实现无法在压缩图中直接找到该节点，于是把整条查询交回原图。

这种做法保证正确，但产生两个问题：

1. 在线索引利用率很低；
2. 区域真实价值被“是否包含高频端点”严重扭曲。

第二版要用真实收益训练模型，所以必须先把这个测量偏差修好，否则标签仍然主要奖励“避开端点”，而不是“覆盖有用的道路需求传播区域”。

2. 什么叫局部接入

局部接入的意思是：端点虽然不在压缩图里，但只需在它所属的 512 节点区域内做一次精确小搜索，把它连接到区域边界，再继续使用压缩图。

它不把内部节点重新永久加入压缩图，也不修改全局索引。边界距离只属于当前查询的临时搜索前沿。

3. 起点是内部节点时

假设起点 S 位于区域 R 内部。查询器在 R 的原始区域子图内运行正向受限 Dijkstra：

$S \rightarrow \text{区域内可达节点} \rightarrow \text{各个可达边界 } B_1、B_2、\dots$

得到：

S 到 B_1 的精确距离
S 到 B_2 的精确距离
...

这些边界及距离共同成为压缩图正向搜索的起始前沿，而不是只选一个最近边界。只选最近边界可能错过经过另一个边界的全局最短路线。

4. 终点是内部节点时

假设终点 T 位于区域 R 内部。需要知道每个边界到 T 的距离。

代码从 T 出发在**反向区域图**中搜索：原图里 $B \rightarrow \dots \rightarrow T$ 的路线，在反图里对应 $T \rightarrow \dots \rightarrow B$ 。因此一次反向受限 Dijkstra 可以得到所有能到达 T 的边界距离。

这些边界与距离成为压缩图反向搜索的初始前沿。

5. 起点和终点都在内部时

如果它们属于不同区域：

- 起点通过所属区域的边界接出；
- 终点通过所属区域的边界反向接入；
- 中间由压缩图双向 Dijkstra 连接。

如果它们属于同一区域，还存在一条可能完全不经过边界的区域内部最短路。查询器因此同时计算：

1. 区域内部 S 到 T 的直达距离；
2. 通过边界和压缩图得到的距离；

最终取较小值。

6. 多前沿双向 Dijkstra

普通双向 Dijkstra 的正向初始状态只有 {起点: 0}，反向只有 {终点: 0}。

局部接入后，初始状态可能是：

正向：{边界 A: 15, 边界 B: 21}

反向：{边界 C: 8, 边界 D: 19}

数值已经包含端点在区域内部走到边界的成本。压缩图搜索从多个带初始距离的边界同时开始，因此叫多前沿搜索。

实现位于 `src/shortest_path.py` 的 `bidirectional_dijkstra_from_frontiers`。

7. 总展开节点如何计算

查询总工作量必须包含：

```
endpoint_access_expanded_nodes  
+ graph_search_expanded_nodes  
= expanded_nodes
```

其中：

- `endpoint_access_expanded_nodes`：区域内局部接入展开；
- `graph_search_expanded_nodes`：压缩图双向搜索展开；
- `expanded_nodes`：两者总和，也是第二版标签使用的量。

8. LRU 缓存是什么

同一个道路节点可能反复作为起点或终点。它到所属区域边界的精确距离不会变化，因此可以缓存。

LRU 是 Least Recently Used，中文可理解为“最近最少使用淘汰”。缓存容量有限时：

1. 每次使用一个条目，就把它标成最近使用；
2. 插入新条目导致超出容量时，删除最久没被使用的条目。

缓存键是：

(节点编号，是否反向，区域编号)

必须包含方向，因为“节点到边界”和“边界到节点”在有向图里不是同一个问题。

9. 为什么标签生成要关闭缓存

缓存会让重复端点查询少做局部搜索，但命中与否依赖：

- 查询执行顺序；
- 之前运行过哪些查询；
- 缓存容量；
- 多进程如何分配任务。

如果标签包含缓存收益，同一个候选换个查询顺序就可能得到不同标签。为了让监督目标只表示区域结构对查询算法的稳定影响，标签生成固定 `endpoint_cache=None`，也就是容量 0。

缓存只在最终在线性能实验中单独评估。

10. 阶段 0 的真实结果

Porto 全量 98,082 条 OD、100 个 512 节点区域上：

- 随机区域和 OD 热点区域距离正确率均为 100%；
- 内部端点整图回退率均为 0%；
- 容量 4,096 的缓存命中率分别为 52.89% 和 62.27%；
- 平均局部接入展开分别减少 47.86% 和 58.33%；

- 同进程配对的平均在线耗时分别减少 1.27% 和 2.20%；
- 缓存开启与关闭的逐查询距离完全一致。

“缓存让耗时下降不多”不代表无效。局部接入只是整个查询的一部分，减少一半局部工作量不一定能减少一半端到端时间。

11. 阶段 0 测试了什么

专项测试覆盖：

- 有向边；
- 起点内部；
- 终点内部；
- 同区域内部直达；
- 跨区域；
- 不可达查询；
- 缓存重复命中；
- 缓存容量与淘汰；
- 缓存开关距离等价。

核心实现位于 `src/indexed_query.py`，相关测试位于 `tests/test_materialized_compression.py`。

12. 阶段 0 为什么不是 GNN 创新

端点局部接入是精确查询引擎的能力。它让任意候选的真实收益测量更公平，也让未来部署的索引更实用。

它没有学习参数，不读取历史 OD 来决定如何搜索，也不产生 GNN 节点表示。因此论文或报告不能把局部接入本身称为“可学习需求传播”。

13. 阶段 0 的阶段门

进入候选与标签阶段前要求：

- 小图测试正确率 100%；
- Porto 全量正确率 100%；
- 内部端点不再整图回退；
- 局部接入与压缩图展开分别计量；
- 缓存只影响性能，不影响距离。

这些条件已经全部满足，所以阶段 0 已关闭。下一章解释固定 1,200 个候选是怎样构建的。